
Data Structures

BS (CS) _Fall_2025

Lab_03 Task



Learning Objectives:

1. 2D and 3D arrays

Task 0: Templates with Arrays (Insert, Delete, Display)

Problem Statement:

Create a template class `Array3D<T>` that stores elements in a 3D dynamic array. The class should support basic insert, delete, and display operations.

Features to Implement:

Constructor

- Takes 3 dimensions (x, y, z) as input.
- Allocates a dynamic 3D array of that size.
- Initializes all values to 0 (or `T {}` default constructor).

`insert(int x, int y, int z, T value)`

- Insert value at position `[x][y][z]`.
- If indices are out of range, show an error message.

`deleteAt(int x, int y, int z)`

- Delete element at `[x][y][z]` by resetting it back to 0 (or `T {}`).
- If indices are out of range, show an error message.

`printArray()`

- Print the entire 3D array in a readable format.

Google Test Cases (GTest)

1. Basic Array Tests (T = int)

- Create `Array3D<int> arr(2, 2, 2)`.
- Verify all elements initialized to 0.
- Insert 42 at (0,1,1) → element at (0,1,1) should be 42.
- Insert value at out-of-range index (e.g., (3,0,0)) → should show error.
- Delete element at (0,1,1) → element should reset to 0.
- Delete element at out-of-range index (2,2,2) → should show error.
- Print array after some insertions → output should contain inserted values.

2. Different Data Type Tests

- `Array3D<double>`
- Create `Array3D<double> arr(2,2,1)`.
- Insert 3.14 at (1,0,0).
- Delete (1,0,0) → value resets to 0.0.

- `Array3D<char>`
- Create `Array3D<char> arr(2,2,2)`.
- Insert 'A' at (0,0,1).
- Delete (0,0,1) → value resets to '\0'.

Task 1: Weather Heatmap Analysis

Problem Statement

A meteorological department records the daily temperature readings of multiple cities. These readings can be stored in a 2D matrix, where:

- Rows = Cities
- Columns = Days
- Entry $A[i][j]$ = temperature of city i on day j

We want to implement a template class `WeatherMatrix<T>` (using a dynamic 2D array) that can perform basic weather data transformations and analysis.

Features to Implement

`rotate90()`

- Rotate the matrix 90° clockwise.
- This changes orientation from cities vs days to days vs cities.
- Useful when displaying temperature heatmaps in rotated form.

`addConstant(int k)`

- Add a constant value k to all entries of the matrix.
- Example: If a thermometer was miscalibrated by $+2^{\circ}\text{C}$, add -2 to correct.

`scale(int factor)`

- Multiply all values in the matrix by a given factor.
- Example: Converting $^{\circ}\text{C}$ readings into tenths of $^{\circ}\text{C}$ (factor = 10).

`dayMax(int dayIndex)`

- Find the maximum temperature recorded on a given day.
- Also identify which city had the hottest temperature on that day.

`printHeatmap()`

- Print the 2D temperature matrix in a clean, grid format.
- Allows visualization of weather data.

GTEST for Weather HeatMap:

```
// Test case for rotate90()
TEST(WeatherMatrixTest, Rotate90) {
    WeatherMatrix<int> wm(2, 3);
    wm.setValue(0, 0, 10);
    wm.setValue(0, 1, 20);
    wm.setValue(0, 2, 30);
    wm.setValue(1, 0, 40);
    wm.setValue(1, 1, 50);
    wm.setValue(1, 2, 60);

    wm.rotate90();

    EXPECT_EQ(wm.getValue(0, 0), 40);
    EXPECT_EQ(wm.getValue(0, 1), 10);
    EXPECT_EQ(wm.getValue(1, 0), 50);
    EXPECT_EQ(wm.getValue(1, 1), 20);
    EXPECT_EQ(wm.getValue(2, 0), 60);
    EXPECT_EQ(wm.getValue(2, 1), 30);
}

// Test case for addConstant()
TEST(WeatherMatrixTest, AddConstant) {
    WeatherMatrix<int> wm(2, 2);
    wm.setValue(0, 0, 5);
    wm.setValue(0, 1, 10);
    wm.setValue(1, 0, 15);
    wm.setValue(1, 1, 20);

    wm.addConstant(5);

    EXPECT_EQ(wm.getValue(0, 0), 10);
    EXPECT_EQ(wm.getValue(0, 1), 15);
    EXPECT_EQ(wm.getValue(1, 0), 20);
    EXPECT_EQ(wm.getValue(1, 1), 25);
}

// Test case for scale()
TEST(WeatherMatrixTest, Scale) {
    WeatherMatrix<int> wm(2, 2);
    wm.setValue(0, 0, 1);
    wm.setValue(0, 1, 2);
    wm.setValue(1, 0, 3);
    wm.setValue(1, 1, 4);
}
```

```

    wm.scale(10);

    EXPECT_EQ(wm.getValue(0, 0), 10);
    EXPECT_EQ(wm.getValue(0, 1), 20);
    EXPECT_EQ(wm.getValue(1, 0), 30);
    EXPECT_EQ(wm.getValue(1, 1), 40);
}

// Test case for dayMax()
TEST(WeatherMatrixTest, DayMax) {
    WeatherMatrix<int> wm(3, 2);
    wm.setValue(0, 0, 25);
    wm.setValue(1, 0, 30);
    wm.setValue(2, 0, 28);
    wm.setValue(0, 1, 22);
    wm.setValue(1, 1, 35);
    wm.setValue(2, 1, 31);

    auto result = wm.dayMax(1); // Day 1 (second column)

    EXPECT_EQ(result.first, 35); // Max temp
    EXPECT_EQ(result.second, 1); // City index
}

// Template reusability check (float matrix)
TEST(WeatherMatrixTest, TemplateFloatMatrix) {
    WeatherMatrix<float> wm(2, 2);
    wm.setValue(0, 0, 1.5f);
    wm.setValue(0, 1, 2.5f);
    wm.setValue(1, 0, 3.5f);
    wm.setValue(1, 1, 4.5f);

    wm.addConstant(0.5f);

    EXPECT_FLOAT_EQ(wm.getValue(0, 0), 2.0f);
    EXPECT_FLOAT_EQ(wm.getValue(0, 1), 3.0f);
    EXPECT_FLOAT_EQ(wm.getValue(1, 0), 4.0f);
    EXPECT_FLOAT_EQ(wm.getValue(1, 1), 5.0f);
}

```

Task 2: University Academic Analytics

Problem Statement

The university wants to store and analyze student marks across multiple semesters and multiple courses.

You are required to implement a template class `AcademicAnalytics<T>` that stores marks in a 3D dynamic array addressed as:

`scores[semester][course][student]`

- Each semester contains multiple courses.
- Each course has marks for multiple students.
- You must allocate, access, and free this structure using raw pointers (`T***`) with `new[]` / `delete[]`.
- No STL containers (e.g., `std::vector`) are allowed for the core storage.

Functional Requirements

Constructor / Destructor

- **Constructor:** `AcademicAnalytics(int semesters, int courses, int students)`
- Dynamically allocate a 3D array (`T***`) and initialize all entries to `T {}` (e.g., 0).
- **Destructor:** Free all allocated memory properly (reverse order).

`setScore(sem, course, student, value)`

- Set the mark of a given student in a given semester and course.

`getScore(sem, course, student)`

- Return the mark at the given indices.

`studentAverage(studentId)`

- Compute and return the average mark of a student across all semesters and all courses.

`courseTopper(courseId)`

- For the given course, compute the total marks of each student across semesters.
- Return the `studentId` of the topper (highest total).
- If there is a tie -> return the lowest `studentId`.

`printSummary()`

Print a simple summary:

- One line per student: Student X average = ...
- One line per course: Course Y topper = student ...

GTEST for University Academic Analytics:

```
// ----- Constructor & Basic Access -----
TEST(AcademicAnalyticsTest, InitializationDefaultsToZero) {
    AcademicAnalytics<int> analytics(2, 2, 3); // 2 semesters, 2 courses, 3
students
    EXPECT_EQ(analytics.getScore(0, 0, 0), 0);
    EXPECT_EQ(analytics.getScore(1, 1, 2), 0);
}

TEST(AcademicAnalyticsTest, SetAndGetScoreWorksCorrectly) {
    AcademicAnalytics<int> analytics(1, 1, 2);
    analytics.setScore(0, 0, 0, 85);
    analytics.setScore(0, 0, 1, 90);
    EXPECT_EQ(analytics.getScore(0, 0, 0), 85);
    EXPECT_EQ(analytics.getScore(0, 0, 1), 90);
}

// ----- Student Average -----
TEST(AcademicAnalyticsTest, StudentAverageSingleStudentMultipleCourses) {
    AcademicAnalytics<int> analytics(2, 2, 1); // 1 student
    analytics.setScore(0, 0, 0, 50);
    analytics.setScore(0, 1, 0, 70);
    analytics.setScore(1, 0, 0, 80);
    analytics.setScore(1, 1, 0, 100);

    EXPECT_DOUBLE_EQ(analytics.studentAverage(0), (50 + 70 + 80 + 100) / 4.0);
}

TEST(AcademicAnalyticsTest, StudentAverageMultipleStudents) {
    AcademicAnalytics<int> analytics(1, 2, 2); // 2 students
    analytics.setScore(0, 0, 0, 40);
    analytics.setScore(0, 1, 0, 60);
    analytics.setScore(0, 0, 1, 90);
    analytics.setScore(0, 1, 1, 70);

    EXPECT_DOUBLE_EQ(analytics.studentAverage(0), (40 + 60) / 2.0);
    EXPECT_DOUBLE_EQ(analytics.studentAverage(1), (90 + 70) / 2.0);
}

// ----- Course Topper -----
```

```

TEST(AcademicAnalyticsTest, CourseTopperSingleSemester) {
    AcademicAnalytics<int> analytics(1, 1, 3); // 3 students
    analytics.setScore(0, 0, 0, 50);
    analytics.setScore(0, 0, 1, 70);
    analytics.setScore(0, 0, 2, 65);

    EXPECT_EQ(analytics.courseTopper(0), 1); // student 1 has highest = 70
}

TEST(AcademicAnalyticsTest, CourseTopperMultipleSemesters) {
    AcademicAnalytics<int> analytics(2, 1, 2); // 2 students across 2 semesters
    analytics.setScore(0, 0, 0, 50);
    analytics.setScore(1, 0, 0, 60);
    analytics.setScore(0, 0, 1, 70);
    analytics.setScore(1, 0, 1, 40);

    // totals: student0 = 110, student1 = 110 -> tie, should return lower ID = 0
    EXPECT_EQ(analytics.courseTopper(0), 0);
}

// ----- Template Reusability -----
TEST(AcademicAnalyticsTest, WorksWithDoubleTemplate) {
    AcademicAnalytics<double> analytics(1, 1, 2);
    analytics.setScore(0, 0, 0, 85.5);
    analytics.setScore(0, 0, 1, 90.2);

    EXPECT_DOUBLE_EQ(analytics.getScore(0, 0, 0), 85.5);
    EXPECT_DOUBLE_EQ(analytics.getScore(0, 0, 1), 90.2);
}

TEST(AcademicAnalyticsTest, WorksWithCharTemplate) {
    AcademicAnalytics<char> analytics(1, 1, 2);
    analytics.setScore(0, 0, 0, 'A');
    analytics.setScore(0, 0, 1, 'B');

    EXPECT_EQ(analytics.getScore(0, 0, 0), 'A');
    EXPECT_EQ(analytics.getScore(0, 0, 1), 'B');
}

// ----- Edge Cases -----
TEST(AcademicAnalyticsTest, StudentAverageAllZero) {
    AcademicAnalytics<int> analytics(1, 2, 1);
    EXPECT_DOUBLE_EQ(analytics.studentAverage(0), 0.0);
}

```

```
TEST(AcademicAnalyticsTest, CourseTopperAllZero) {  
    AcademicAnalytics<int> analytics(1, 1, 2);  
    EXPECT_EQ(analytics.courseTopper(0), 0); // tie, lowest ID  
}
```